Studio Nx

JS 270 — 되돌리기 · 다시 하기

2026-10-19

JavaScript 270 — 되돌리기 · 다시 하기 (`sequencer-history.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-history.js`
작업	S1-10 프로젝트 전체 저장·불러오기: Undo/Redo (PRD 02 FR-255)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	269 명령 레지스트리 · 271 저장 · 자동 저장

1. 하는 일

디바이스(악기·이펙트)를 바꾼 동작을 기억하고, Ctrl+Z / Ctrl+Shift+Z 로 앞뒤로 넘깁니다. 최대 100단계까지 기억합니다.

```
pushHistory({ label, undo, redo })
        ↓
stack = [ 단계1, 단계2, …, 단계100 ]   ← at = 지금 위치
        ↓
undo()  → stack[at-1].undo()  → at -= 1
redo()  → stack[at].redo()    → at += 1
        ↓
applying = true 동안에는 새 이력을 쌓지 않습니다
undo · redo 가 Promise 를 돌려주면 끝날 때까지 applying 을 켜 둡니다 (서버 저장이 끝난 뒤 풉니다)
```

2. 언제 쓰나

쓰는 곳	
`sequencer.js` `handleChange`	디바이스 추가·고치기·떼기·순서 바꾸기, BPM·Key 변경마다 `pushHistory` 를 부릅니다
메뉴 EDIT	`edit.undo` · `edit.redo` 명령으로 실행합니다
Ctrl+Z · Ctrl+Shift+Z · Ctrl+Y	`sequencer.js` `initShortcuts` 가 여기 `undo` · `redo` 를 부릅니다
커맨드 팔레트	`runCommand('edit.undo')` 로도 실행됩니다

3. 코드

```
export const HISTORY_LIMIT = 100;
let stack = []; let at = 0; let applying = false;
let onStateChange = function () {}; let toast = function () {};

export function initHistory(options) {
  const opts = options || {};
  if (typeof opts.onStateChange === 'function') onStateChange = opts.onStateChange;
  if (typeof opts.toast === 'function') toast = opts.toast;
  registerCommands({ 'edit.undo': undo, 'edit.redo': redo }, { group: 'edit' });
  notifyState();
}
```

```
export function pushHistory(entry) {
  if (applying || !entry || typeof entry.undo !== 'function' || typeof entry.redo !== 'function') return
false;
  stack = stack.slice(0, at);
  stack.push(entry);
  if (stack.length > HISTORY_LIMIT) stack = stack.slice(stack.length - HISTORY_LIMIT);
  at = stack.length;
  notifyState();
  return true;
}

export function undo() {
  if (!canUndo()) { toast('Nothing to undo'); return false; }
  const entry = stack[at - 1];
  applying = true;
  const done = function () { applying = false; at -= 1; notifyState(); };
  try {
    const result = entry.undo();
    if (result && typeof result.then === 'function') result.then(done, done);
    else done();
  } catch (err) { done(); }
  return true;
}

export function redo() {
  if (!canRedo()) { toast('Nothing to redo'); return false; }
  const entry = stack[at];
  applying = true;
  const done = function () { applying = false; at += 1; notifyState(); };
  try {
    const result = entry.redo();
    if (result && typeof result.then === 'function') result.then(done, done);
    else done();
  } catch (err) { done(); }
  return true;
}
```

문법 풀이

식	뜻
`stack.slice(0, at)`	되돌린 자리 뒤는 버립니다. 새로 바꾼 것을 기록하면 이전 되돌림 경로가 사라집니다
`applying = true` + `done()`	undo 를 실행하는 동안에 들어오는 알림(새 저장)을 무시합니다. 실패해도 반드시 풉니다
`result.then(done, done)`	디바이스 고치기는 서버 PATCH 라서 Promise 를 돌려줍니다. 그 저장이 끝난 뒤에 `applying` 을 풀고 위치를 옮깁니다. 저장 중에 오는 알림이 이력을 더럽히지 않습니다 (2026-10-19)
`stack.length > HISTORY_LIMIT`	100단계를 넘기면 가장 오래된 것부터 버립니다
`notifyState()`	메뉴의 `edit.undo` · `edit.redo` 항목을 지금 상태에 맞게 켜고 끕니다

4. 왜 이렇게

선택	이유
undo/redo 를 클로저로 담음	같은 동작을 다시 만들지 않고, 이전 값·이후 값을 함께 담습니다
100단계로 제한	FR-255 가 최소 100단계를 요구합니다. 메모리에 과하지 않게 상한을 둡니다
`applying` 깃발	undo 가 만든 알림이 다시 pushHistory 로 들어가면 이력이 어긋납니다. 디바이스 고치기는 저장이 서버로 다녀오는 동안에도 켜 두어야 하므로, Promise 가 끝난 뒤에 풉니다 (2026-10-19)
`canUndo()` · `canRedo()` 를 export	시험과 메뉴가 현재 상태를 알 수 있습니다
`historyLabels()` 는 시험용	어느 순서로 쌓였는지 눈으로 확인할 수 있습니다

5. 시험

`docs/handoff/s1-10_tests/ui.mjs` 에서 이펙트 노브를 한 번 돌린 뒤 Ctrl+Z → 값이 이전으로 돌아가고, Ctrl+Shift+Z → 다시 이전 값이 되는지, `canUndo()` · `canRedo()` 가 그에 맞게 바뀌는지, 100단계를 넘기면 `historyDepth()` 가 100을 넘지 않는지 확인합니다.